

Software Implementation and Testing Document

For

Group <RuneScape Web App>

Version 1.0

Authors:

Agustin Camus
Colton Bregoff
Miles Burkart
Ricardo Furst
William Kilduff

1. Programming Languages (5 points)

RuneReport is a static, client-side web application with no backend, so the whole project is written in the three standard languages of the web:

- **HTML** - Defines the structure of every page. Each page contains the shared navigation bar, the page title, and the page-specific controls such as the Browse search box.
- **CSS** - Handles all styling: the dark-brown and gold theme, the navigation bar, and the flexbox layouts for the Browse controls and item list. It also contains the keyframe animations for the RuneScape-style text effects.
- **JavaScript** - Provides all client-side behavior. Currently this includes the text-effect compiler and the per-page script. In later increments it will fetch and process market data, run the strategy backtests, and render reports.

We chose these languages because the project is a client-side web application that needs no server, which keeps development and deployment simple. JavaScript lets the pages work with market data and run the strategy tests directly in the browser, and it is the standard way to do so. Several team members were new to JavaScript, but we chose it for the project's needs and are learning it as we go.

2. Platforms, APIs, Databases, and other technologies used (5 points)

Technology	Type	Use in the project
OSRS Wiki Real-time Prices API	External API	Source of Old School RuneScape Grand Exchange item and historical price data. It feeds the Browse page item list, the item pages, and the backtesting engine that replays user-defined strategies against historical prices
OsFont (os-font.js / os-font.css)	Front-end library	A small js and CSS helper that applies RuneScape chat-style text effects (colors, flash, glow, wave, shake, scroll, slide) to paragraphs with prefixes such as flash1:wave:. It is loaded on every page and runs with OsFont.compile(). It is third-party code included in the repository under fonts.
RuneScape Chat Font	Font asset	Provides the in-game look for headings and effect text. It is packaged in fonts/font/ as a @font-face kit and used by the page title and OsFont effects.
GitHub (7Limes/CEN4090L-Project)	Version control / hosting	Hosts the source code and the project documents. Instructor and TAs have access to this public repository.
GitHub Issues	Project Tracking	Tracks features and bugs, grouped by increment. Every pull request is linked to an issue.
Discord	Communication	Team calls and coordination outside of class meetings.
Web browser (client)	Platform	The application runs entirely in the user's browser. No server-side code or user accounts exists; all users are guests.

Databases: This project uses no database. All data is retrieved from the OSRS Wiki Real-time Prices API on demand. Saved reports (shown on the "Saved Reports" page) are stored on the client side only. We will describe the storage mechanism in more detail once the saving feature is implemented.

3. Execution-based Functional Testing (10 points)

This is to be completed for Increment 2. At this stage, the Home, Browse, item, report, and saved reports pages have been checked manually in the browser during development. A formal functional test plan covering the requirements listed in the RD will be added in the next increment.

4. Execution-based Non-Functional Testing (10 points)

This is to be completed for Increment 2. Non-functional testing against the requirements in the RD (for example page load time, API response handling, and browser compatibility) will be described in the next increment.

5. Non-Execution-based Testing (10 points)

To be completed for Increment 2. Code reviews of pull requests before merging in the main branch will be documented here in the next increment. For this increment pull requests after completing issues were merged to main without any problems.